

协同办公产品竞品分析

飞书 vs 钉钉：管控效率与用户体验的平衡

版本	v1.2
作者背景	建筑公司实习经历，观察过工地场景下的协同工具使用
分析维度	信息架构 · 交互设计 · 流程自动化 · AI 能力
日期	2026-05-31

阅读导航

如果你只有 5 分钟	读 1.2 「一句话核心发现」 + 第七章 「设计启示」
如果你有 15 分钟	加上 4.3 「工地巡检的两种方案」 + 6.3 「房建工人的一天」
贯穿全文的命题	B 端产品如何在「管控效率」和「用户体验」之间找到平衡？

目录

代码块

1	一	分析框架与核心发现
2	1.1	分析框架：四个维度
3	1.2	一句话核心发现
4	1.3	产品定位速览
5		
6	二	信息架构对比
7	2.1	导航结构
8	2.2	信息中枢：文档 vs 审批
9	2.3	场景对比：一条需求通知的两种信息流
10		
11	三	交互设计对比
12	3.1	Thread 与 DING
13	3.2	已读回执：做与不做的选择
14	3.3	新成员入群：历史可见 vs 历史隔离
15	3.4	小结：两种交互取向
16		
17	四	流程自动化对比
18	4.1	审批引擎：基础够用 vs 复杂极致
19	4.2	低代码平台：多维表格 vs 宜搭
20	4.3	场景还原：工地安全巡检的两种数字化方案
21	4.4	分析：强触达机制的场景适配性
22		
23	五	AI 能力对比
24	5.1	飞书智能伙伴：嵌入工作流的 AI
25	5.2	钉钉 AI 助理：行业化的 Agent
26	5.3	边界变化：飞书补管控，钉钉补体验
27	5.4	AI 作为管控与体验的调和层
28		
29	六	功能矩阵与用户场景还原
30	6.1	功能对比总览
31	6.2	产品经理的一天
32	6.3	房建工人的一天
33		
34	七	设计启示：管控与体验的平衡
35	7.1	为什么这道题难
36	7.2	三条设计原则
37	7.3	改进方案：审批通知的柔性化改造
38	7.4	我对产品设计的理解

39

40

41

42

附录

附录 A：参考资料

附录 B：分析局限性

一 分析框架与核心发现

1.1 分析框架：四个维度

大多数飞书 vs 钉钉的对比止步于功能罗列——你有 Thread、我有 DING、你有多维表格、我有宜搭。但功能列表回答不了核心问题：**两款产品在同一个功能上做出相反设计时，各自的依据是什么？**

比如「已读回执」——飞书不做，钉钉做到每条消息可追踪已读人员。这显然不是技术差距。要理解这种分歧，需要穿透功能表层，进入产品设计的底层逻辑。

我选取了四个维度：

维度	关注的问题
信息架构	信息如何组织？入口在哪、路径多长？这决定了用户每天接触产品的第一感受
交互设计	关键操作节点上，产品如何回应？这暴露了产品对用户的默认预期
流程自动化	哪些重复性工作被自动化了？自动化的边界画在哪里？这决定了产品的能力上限
AI 能力	AI 是独立功能还是嵌入 workflow？它正在改变什么？这是当

	前变化最快的变量
--	----------

四个维度层层递进：信息架构决定交互起点，交互设计决定流程效率，流程自动化的下一步是 AI。贯穿四个维度的问题始终是一个：**这个设计是在增强管控效率，还是在提升用户体验？两者是否必须非此即彼？**

本报告中的工地场景来自我在某建筑科技公司的实习观察。这段经历让我意识到：同一款产品在办公室和工地上，「好用」的标准可能完全相反。

1.2 一句话核心发现

飞书以「文档+会议」为信息中枢，追求知识沉淀与异步协同，交互体验流畅；钉钉以「审批+已读」为管控核心，强在组织层级穿透力。两者产品边界正在 AI 能力上发生重合——飞书的 Agent 开始承担任务追踪功能，钉钉的 AI 助理开始提供群聊摘要和文档润色。AI 正在成为管控与体验之间的调和层：让管理者获取需要的确定性，同时让使用者不感到被审视。

1.3 产品定位速览

维度	飞书	钉钉
母公司	字节跳动	阿里巴巴
上线时间	2019 年	2015 年
公开用户量	未披露	超 7 亿用户 / 2,500 万+组织 (2024 官方数据)
定价模式	按人头（专业版 ≈¥200/人/年）	按组织（专业版 ≈¥9,800/年，不限人数）
信息中枢	文档 + 会议	审批 + 已读
产品隐喻	乐高——零件不多，组合灵活	瑞士军刀——功能全面，开箱即用

二 信息架构对比

2.1 导航结构

飞书桌面端：左侧导航栏 5 个一级入口——消息、日历、文档、会议、工作台。每个入口下不超过 5 个二级项。新员工打开飞书，一眼看完所有路径。

钉钉桌面端：底部 Tab 栏 4 个入口（消息、文档、工作台、通讯录），顶部另有搜索栏和快捷入口。工作台内有数十个应用图标。新员工打开钉钉，需要时间建立对功能布局的认知。

对比项	飞书	钉钉
一级导航数量	5 个	4 Tab + 顶部栏
首屏信息密度	低	高
新用户认知负荷	低	中-高
功能发现方式	渐进式——需要时出现	罗列式——首屏展示全貌

飞书假设用户打开 App 时有明确目的——发消息、看日历、写文档——因此把高频入口放在门口，低频功能藏到深处。钉钉假设用户可能不知道有哪些功能可用——因此尽可能在首屏展示更多选项。

这两种设计对应两种用户：知道自己要什么的人，和需要被告诉有什么可用的人。

2.2 信息中枢：文档 vs 审批

两款产品信息架构的根本差异在于「信息的重心放在哪里」。

飞书以文档为中枢：

代码块

- 1 消息（讨论）→ 文档（结论沉淀）→ 知识库（归档）→ 搜索 / AI 检索
- 2 会议（讨论）→ 妙记转录为文字文档 → 同样进入可搜索的知识库

一切讨论的终点是文档。消息里的结论写入文档，会议录音转为文档，知识库是文档的集合。文档是信息在飞书中的最终形态——可搜索、可复用、可沉淀。

钉钉以审批为中枢：

代码块

- 1 消息（通知）→ DING（确保触达）→ 审批（通知变成决策）→ 归档留痕
- 2 考勤（行为记录）→ 报表（汇总为管理数据）→ 进入管理者视野

一切消息的终点是审批。通知是为了发起审批，DING 是为了催审批，考勤数据最终变成管理者审批排班和绩效的依据。审批是信息在钉钉中的最终形态——可追溯、可管控、不可篡改。

这个差异不是偶然的，它解释了双方几乎所有的功能投入优先级：飞书在文档上投入最多（块编辑器、多维表格、知识库、AI 写作），因为文档是它的核心；钉钉在审批上投入最多（复杂条件分支、并行审批、DING 催办、行业模板），因为审批是它的核心。

2.3 场景对比：一条需求通知的两种信息流

场景：产品总监要将「下周三评审会」的时间变更通知全团队。

飞书的信息流：

代码块

- 1 总监在飞书文档里修改评审会时间
- 2 ↓
- 3 文档 @ 了所有参会人 → 每人收到一条通知
- 4 ↓
- 5 有人对时间有疑问 → 在文档里划词评论（不干扰群聊消息流）
- 6 ↓
- 7 总监在评论串里回复 → 所有人可见
- 8 ↓
- 9 有人事后搜索「评审会」→ 找到文档 → 看到最新时间和全部讨论

信息的真相源只有一个：那个文档。所有人围绕文档协作，讨论附着在文档上。事后任何人来查，都能获取完整上下文。

钉钉的信息流：

代码块

- 1 总监在群里发消息：「评审会改到周三下午 2 点」
- 2 ↓
- 3 DING 全员 → 手机震动 + 短信提醒 → 确保每个人被触达
- 4 ↓
- 5 同事 A：「收到」
- 6 同事 B：「会议室订了吗」
- 7 总监：「订了 3 号」
- 8 同事 C：「我周三下午有会，能否改到 4 点」
- 9 同事 D：「4 点我可以」
- 10 （讨论产生 15 条消息）
- 11 ↓
- 12 周三有人忘记时间 → 翻聊天记录 → 翻找 2 分钟 → 再问群里确认

信息的真相源是碎片化的，散落在十几条消息里。事后想回溯，需要人工翻找。

同一个需求，飞书用「一个文档」承载信息，钉钉用「消息流」承载信息。前者把信息变成可复用的知识，后者把信息当作一次性指令。这不是优劣之分——如果信息需要反复查阅（如项目计划），文档中枢更高效；如果信息只需要即时传达（如临时通知），消息流更直接。

三 交互设计对比

3.1 Thread 与 DING

这两项功能代表了双方在 IM 交互上最根本的分歧。

飞书的 Thread（话题串）：

群聊中，长按某条消息 → 点击「回复」→ 弹出 Thread 面板。所有针对这条消息的回复聚合在 Thread 内，不流入主消息流。未参与该话题的人，消息流不受干扰。

效果：群聊主消息流保留「信噪比」——重要的新消息不会被讨论淹没。Thread 适合异步协作：你不用实时关注每条讨论，有空时再进入 Thread 查看。

钉钉的引用回复：

群聊中，长按某条消息 → 点击「引用」→ 在主消息流中插入一条带引用块的回复。所有引用回复都在主消息流里。同一张设计稿的讨论产生 5 条引用回复，穿插在主消息流中，把不相关的讨论（「下午评审会谁参加」）冲到屏幕外。

效果：消息流是全部信息的唯一通道——讨论和通知共享一条流。引用回复适合实时沟通：你不会漏掉任何讨论，但你需要承受更高的信息噪音。

与引用回复配合的，是钉钉独有的 **DING**——强制触达机制。发送者可以通过短信和电话双通道提醒接收者查看消息，确保指令必达。飞书没有等价功能。

差异来源：飞书将「讨论」和「通知」分离——Thread 承载讨论，消息流承载通知，互不干扰。钉钉将「讨论」和「通知」合一——一切都在消息流里，DING 确保重要消息不被遗漏。前者适合知识型团队的异步工作方式，后者适合执行型团队的指令传达需求。

3.2 已读回执：做与不做的选择

飞书不支持已读回执。钉钉支持单聊已读和群聊每条消息的已读/未读人员列表。

这不是技术选择，是产品对用户关系的预设。

飞书的逻辑：消息的阅读状态属于接收者。你发消息给我，我什么时候看、什么时候回，是我的自主权。已读回执会制造无形的催促——「你看了为什么不回」。

钉钉的逻辑：消息的阅读状态属于发送者。我发消息给你，我需要确定性——你收到了没有。如果你已读但长时间不回复，说明需要跟进。这是管理动作的基础。

两种逻辑适配不同的组织形态：知识型团队中，同事是平等协作者，已读回执带来社交压力而非效率。执行型团队中，管理者需要确认指令覆盖率，已读回执是管理工具而非社交工具。

3.3 新成员入群：历史可见 vs 历史隔离

一个新员工加入部门群后——

飞书：她能看到入群前的全部历史消息。可以自己往上翻，了解团队近期讨论。

钉钉：她只能看到入群之后新发的消息。入群前的消息全部不可见。

	飞书	钉钉
新员工感受	「我可以自己补课」	「我需要问别人」
传递的信号	你被信任，可以自主获取信息	信息按权限分层分发
对主动型员工	友好	尚可——会主动询问

对被动型员工	友好——不求助也能获取上下文	不友好——不问就不知道
--------	----------------	-------------

钉钉完全可以把历史消息设为默认可见——这只是一个默认值。它选择不做，因为它的信息架构假设是「信息按层级分发」：新成员处于组织外围，不应接触入群前的内部讨论。飞书的假设相反：「信息是团队的共同资产」：新成员是团队的一员，理应能获取上下文。

3.4 小结：两种交互取向

维度	飞书	钉钉
消息组织	Thread 分离讨论与通知	引用回复 + DING，全部在主消息流
阅读状态	不追踪	精确追踪
新成员信息获取	默认可获取历史	默认隔离历史
底层预设	信任用户	管控流程

两款产品在每一个交互节点上的选择，都可以追溯到同一个底层预设：飞书默认用户是自驱的协作者，钉钉默认用户需要被管理和引导。

四 流程自动化对比

4.1 审批引擎：基础够用 vs 复杂极致

审批是钉钉起家的功能，也是双方差距最大的模块。

一个对比案例：合同审批——金额超过 5,000 元且类型为采购合同时，需直属上级和财务总监同时审批，两人都通过才算完成，任何一人驳回则退回重填，超过 24 小时自动催办。

飞书能做到：员工提交 → 金额 > 5,000 → 抄送财务总监 + 上级审批 → 完成。仅支持简单条件分支，不支持并行审批（两人同时审）。

钉钉能做到：员工提交 → 金额 > 5,000 且类型为采购合同 → 上级 + 财务并行审批 → 两人都通过 → 自动抄送法务 → 完成。任一驳回 → 退回员工重填。超时 24h → 自动 DING 催办。

审批能力	飞书	钉钉
基础表单（请假、报销）	✓	✓
单条件分支	✓	✓
多级嵌套条件	✗	✓
并行审批（多人同时）	✗	✓
会签 / 或签	✗	✓
DING 催办	✗	✓
行业模板	约 30+	数百+
审批-支付打通	✗	✓（支付宝）

飞书审批覆盖约 80% 通用场景。剩下 20% 的复杂场景（合同、采购、离职流程），钉钉是唯一选择——而这 20% 恰恰是企业最刚性的管理需求。

4.2 低代码平台：多维表格 vs 宜搭

飞书多维表格：本质是零代码数据库。用户定义字段（文本/数字/日期/附件/关联），然后一键切换视图——表格、看板、甘特图、日历、表单。同一份数据，不同视角。适合需求跟踪表、内容日历、轻量客户管理。业务人员能自学使用。

钉钉宜搭：本质是低代码应用搭建平台。用户拖拽表单、列表、图表、流程组件，搭建完整业务应用，可发布到工作台。天然打通钉钉审批引擎和通讯录。适合进销存系统、巡检应用、教务管理。需要 IT 人员或资深业务人员配置。

	飞书多维表格	钉钉宜搭
上手门槛	低——会 Excel 就会用	中——需理解组件和流程
功能上限	轻量业务管理	完整业务系统
与审批打通	弱	强
模板生态	有限	丰富，行业化
核心用户	业务人员自建	IT 搭建后分发

4.3 场景还原：工地安全巡检的两种数字化方案

以下场景来自我在某建筑科技公司实习期间的观察。工地安全管理的核心痛点：巡检发现隐患后，如何在最短时间内将整改指令传达到一线工人并确保执行。

场景：安全员巡检时发现 4 号楼 8 层临边防护缺失。拍照取证，需通知施工班组整改。

方案 A：基于飞书多维表格

代码块

1

安全员打开飞书移动端 → 拍照

2

↓

3

打开多维表格「安全巡检记录」 → 新增一行

4

↓

5

填写字段：位置/问题描述/严重等级/责任人/限期整改时间

6

↓

7

多维表格自动化：「严重等级=高」

8

→ 自动通知项目经理 + 安全总监

9

↓

10

施工班长在飞书收到通知 → 点开记录 → 查看照片和整改要求

```
11      ↓
12  整改后拍照 → 在同一行更新状态=「已整改」+ 上传整改后照片
13      ↓
14  状态变更触发自动化 → 通知安全员复查
15      ↓
16  安全员现场确认 → 状态改为「已关闭」→ 记录归档
```

方案 A 特点：轻量、灵活、数据可查询。安全员无需 IT 支持即可搭建。但有一个关键前提——**施工班长需要主动查看飞书通知并执行**。如果班长在施工中忽略了通知，整个流程中断。

方案 B：基于钉钉宜搭

代码块

```
1  安全员打开钉钉移动端 → 拍照
2      ↓
3  打开宜搭「安全巡检」应用 → 新增巡检单 → 提交
4      ↓
5  宜搭流程自动触发：严重等级=高 → 发起整改审批流
6      ↓
7  审批流：项目经理审批 → 自动生成整改工单 → DING 施工班长
8      ↓
9  DING 通过短信+电话+App 三通道触达 → 班长无法忽略
10     ↓
11  班长在宜搭更新整改状态 + 上传照片
12     ↓
13  系统自动追踪：超时 4h 未整改 → 自动升级 DING 到项目经理
14     ↓
15  整改完成 → 安全员复查确认 → 审批流关闭 → 全部记录归档
```

方案 B 特点：闭环、强制、超时自动升级。但需 IT 人员在宜搭中配置完整应用和审批流。安全员每次巡检需填写 15+ 字段的表单，部分字段与当前场景无关。

两种方案的差距不在功能多少，而在对执行者的预期不同。 方案 A 假设班长会主动关注通知并执行；方案 B 不依赖这种假设——用 DING 确保触达，用超时升级兜底。

4.4 分析：强触达机制的场景适配性

在办公室场景中，飞书的「轻量灵活」是优势。但在我实习的工地上，它反而是劣势。

原因很具体：一线建筑工人的手机主要用于打电话和刷短视频，不会保持飞书在线。安全员发出一条飞书通知，工人可能 2 小时后才看到。在「临边防护缺失」这种场景下，2 小时的延迟意味着事故风险。

钉钉的 DING——短信 + 电话 + App 三通道触达——恰好解决了这个问题。不管你用不用 App，短信和电话一定会被注意到。超时自动升级机制也契合安全管理的刚性需求：整改不能依赖个人主动性，必须有系统兜底。

但钉钉方案的问题同样明显：安全员每次巡检都需填写冗长表单，字段多为 IT 部门统一配置，与当前场景匹配度低。一个更合理的设计应该是：保留钉钉的强制触达和闭环追踪，但允许安全员自定义表单字段，像飞书多维表格那样轻量配置。

这个场景恰好体现本报告的核心命题：管控要做，但不能以牺牲操作体验为代价。两者的平衡点，取决于场景对「确定性」的需求程度。在安全场景下，管控优先；在日常协作场景下，体验优先。

五 AI 能力对比

5.1 飞书智能伙伴：嵌入工作流的 AI

飞书智能伙伴不是飞书内部的一个独立对话机器人。它的入口不在某个固定位置，而是分布在用户正在做的事情里：

- 在文档中选中文字 → 直接润色、续写、翻译——不需要打开 AI 对话框
- 会议结束 → 妙记自动转录 → AI 自动出纪要和待办 → 自动发送到会议群——用户零操作
- 搜索框输入「上周关于支付的讨论」→ 返回消息片段、文档段落、会议录音——跨模块语义搜索

- Agent 2.0：可跨文档、表格、日历、审批执行多步骤任务——「汇总本周需求变更 → 生成周报 → @ 责任人确认」

飞书的 AI 路径：不是让用户去找 AI，而是让 AI 在用户正在做的事情里出现。底层模型为字节自研豆包大模型，迭代节奏不受外部厂商约束。

5.2 钉钉 AI 助理：行业化的 Agent

钉钉 AI 助理有独立的对话入口，也可嵌入审批、宜搭、考勤等模块。它最独特的能力是行业 Agent：

- 制造业 Agent：读取 MES 系统数据，分析产线效率，预警产能瓶颈
- 教育 Agent：辅助排课、成绩分析、家校通知生成
- 政务 Agent：公文处理、会议纪要、政策问答

钉钉的 AI 路径：将 AI 引入每个行业的业务流程。底层模型为阿里自研通义千问。

5.3 边界变化：飞书补管控，钉钉补体验

AI 出现之前，飞书做「体验」、钉钉做「管控」，边界清晰。

AI 正在模糊这条边界：

- 飞书的 Agent 2.0 开始承担任务追踪和流程管理——「汇总需求变更 → 生成周报 → 跟踪负责人确认」。这本质上是管控功能，但以 Agent 对话的方式呈现，用户不感到被管理。
- 钉钉的 AI 助理开始提供群聊摘要、文档润色、会议纪要——这些传统上属于飞书的体验长板，钉钉通过 AI 在补齐。

飞书从「体验」出发，用 AI 补「管控」；钉钉从「管控」出发，用 AI 补「体验」。双方没有直接复制对方的功能（飞书没有做 DING，钉钉没有做 Thread），但都在用 AI 覆盖对方的核心能力。

5.4 AI 作为管控与体验的调和层

这是本次分析最重要的发现。

以前，「管控」和「体验」是零和关系：要做管控，就得加已读回执、加 DING、加审批节点，用户体验必然受损；要做体验，就得去掉这些，管理者的确定性必然下降。

AI 提供了第三条路：

- 不需要已读回执 → AI 分析群聊参与度，向管理者报告「关键人员都已参与讨论」
- 不需要 DING 轰炸 → AI 识别消息紧急程度，仅对真正紧急的消息强提醒
- 不需要复杂审批流 → AI 预判审批路径，简化人工操作

AI 让管控从「显性规则」变成「隐性感知」——管理者获得需要的确定性，使用者不感到被审视。这是 B 端产品设计正在发生的变化：管控和体验不再是二选一，而是可以被 AI 调和的两个目标。

六 功能矩阵与用户场景还原

6.1 功能对比总览

功能模块	飞书	钉钉	一句话差异
IM	Thread 话题串，无已读	引用回复，已读精确到人，DING	异步协作 vs 实时管控
文档	块编辑器+多维表格+知识库	线性编辑器+多维表	飞书代际领先
会议	妙记转录+AI 纪要	闪记转录+AI 纪要	趋同，飞书略优

审批	基础够用，模板 30+	复杂引擎，模板数百+	钉钉碾压级领先
低代码	多维表格（零代码）	宜搭（低代码平台）	宜搭成熟度领先
AI	嵌入工作流，原生化	行业 Agent，普惠化	飞书更深，钉钉更广
考勤	基础打卡	GPS/WiFi/蓝牙/人脸	钉钉碾压级领先
开放平台	开发者文档行业标杆	ISV 数量多，行业应用丰富	各有优势

6.2 产品经理的一天

角色：小陈，SaaS 公司产品经理，桌面办公为主。

时间	动作	飞书	钉钉
08:30	看未读	AI 推送未读摘要：3 个群共 47 条的要点	逐群翻看，新入的群看不到历史
09:00	写 PRD	文档+内嵌多维表格跟踪需求+AI 润色	文档+手动建任务清单
10:00	讨论设计稿	Thread 回复，主消息流不受影响	引用回复，消息流被讨论刷屏
14:00	评审会	会议→共享文档批注→妙记纪要→自动发群	会议→文档→闪记纪要→被后续消息淹没
16:00	搜索信息	跨文档+消息+会议语义搜索	消息和文档分开搜索
整体感受	信息在一个系统内流转，不丢失	功能全但模块间有缝隙，需手动搬运信息	

6.3 房建工人的一天

角色：老李，42 岁，房建项目木工班长，初中文化。手机主要用于打电话和刷短视频。工作需要：打卡、接任务、报完工。不写文档。

时间	动作	飞书	钉钉
05:50	到工地打卡	飞书基础打卡（GPS）	GPS+WiFi+人脸，到工地门口自动弹出打卡页
06:00	班前安全交底	群发飞书文档链接→老李点开看→回复「收到」	群发消息+图片→已读显示 28/30 人→没读的 2 人被 DING
07:30	申领材料	打开多维表格→填木方数量规格→提交	打开宜搭应用→扫码选规格→自动通知材料员和仓库
10:00	安全巡检	安全员发多维表格记录→老李收到飞书通知→正在施工没看手机→1h 后看到	安全员发宜搭单→DING 老李（短信+电话）→被迫停下手里的活→马上整改
14:00	隐蔽验收	拍照上传多维表格→监理在飞书收到通知→审批	拍照上传宜搭→自动发起审批→监理审批→DING 通知下一环节→全部留痕
17:00	下班	飞书打卡→手动填日报（「4 号楼 8 层模板完成」）	钉钉打卡+人脸→日报自动汇总当天工单数据→老李仅需确认
感受	「挺简单，但通知有时候看不到。那个表格字有点小。」	「东西多，刚开始不会用。但 DING 一来就知道了，不会耽误事。」	

同一款工具，在不同人手里完全不同。老李不需要文档协作和异步讨论，他需要打卡不出错、任务不漏掉、通知一定能收到。飞书的「轻量灵活」在他身上变成「通知可能漏掉」。钉钉的「强制触达」在他身上变成「虽然有点烦但不耽误事」。在工地场景下，管控效率优先于操作体验——因为安全不等人。

七 设计启示：管控与体验的平衡

7.1 为什么这道题难

B 端产品的核心矛盾：管理者要管控，使用者要自由。两者天然冲突。

管理者要的	使用者要的
确定性——消息发出去，你看到没有	自主权——什么时候回是我的事
可追踪——流程走到哪，谁卡住了	不被盯着——别每个动作都监控
执行力——整改必须今天完成	灵活性——我有自己的工作节奏

飞书偏向使用者一侧：去掉已读回执、不做 DING、新成员可见历史。代价是管理者觉得「管不住人」。

钉钉偏向管理者一侧：已读精确到人、DING 强制触达、审批引擎做到极致。代价是使用者觉得「被管着不舒服」。

不能简单说谁对谁错——需要回答的是：**在什么场景下，管控应该让步给体验？在什么场景下，体验应该让步给管控？**

7.2 三条设计原则

基于本次分析，我提炼了三个可以在产品设计中复用的原则：

原则一：分级，而不是一刀切。 不是所有消息都值得强提醒。安全警告和日报通知不应使用同一等级。给发送者提供通知分级选项——紧急用强触达，重要用推送，普通用红点。把选择权给发送者，

把安宁感给接收者。

原则二：用 AI 做调和层。 管理者需要的不是「已读回执」，而是「我的消息是否触达了关键人员」。AI 可以分析群聊参与度和消息互动情况，生成触达报告给管理者，而不是用已读标记催促每个接收者。管控目标达成，但管控手段隐形。

原则三：不同场景，不同默认值。 同一个产品，一个项目讨论群和一个紧急通知群应有不同的默认行为。允许群管理员定义群类型——讨论型默认关闭已读回执、开启 Thread；指令型默认开启已读追踪和强触达。不做全局开关，让每个场景有合适的默认值。

7.3 改进方案：审批通知的柔性化改造

以钉钉审批通知为例，演示原则一「分级」的落地。

当前问题：钉钉的审批通知全部使用 DING 强触达。审批人每天被 DING 数十次，产生「DING 疲劳」——紧急审批和常规审批混在一起，反而削弱了强提醒的效果。

改进方案——审批通知按紧急程度分级：

级别	触发条件	通知方式	示例
● 紧急	安全整改、超时升级、合同到期	DING 全部通道（短信+电话+App）	「4 号楼防护缺失，2h 内必须整改」
● 重要	请假、报销、常规采购	App 推送 + 消息列表置顶，不电话	「员工请假申请，24h 内处理」
● 普通	日报审批、知会型流程	消息列表红点，不推送	「日报已提交，有空再看」

设计逻辑：不是削弱管控，是让管控更精准。把强提醒资源集中在真正紧急的审批上。使用者的打扰感下降，管理者的紧急审批响应率反而上升——因为它不再被淹没在大量常规 DING 中。管控和体验同步提升，而非此消彼长。

7.4 我对产品设计的理解

在工地实习时，我注意到一个现象：最好的安全员不是罚单开得最多的，而是工人愿意主动跟他报告隐患的。工人碰到他，会说「老张，4 号楼那个架子有点晃」。这比他一个人巡检发现隐患快得多。

做 B 端产品也是同理。最好的管控不是让用户感到被管——已读标记盯着他、DING 追着他、审批卡着他——而是让用户愿意配合。他主动更新任务状态，因为知道这是为了团队协作而非被监控。他主动响应通知，因为收到的每条通知都值得看，而非被滥发。

管控效率与用户体验的平衡，不是一道选择题——不是选管控还是选体验。而是一道设计题——用什么方式达成管控目标，同时让使用者不感到被冒犯。

AI 让我对这道题的未来感到乐观。因为 AI 可以把管控从「显性的规则」变成「隐性的感知」。这不是技术乐观主义——飞书的 Agent 和钉钉的 AI 助理已经在这条路上走出了第一步。

附录

附录 A：参考资料

来源	用途
飞书官网及帮助中心	产品功能、交互验证
钉钉官网及帮助中心	产品功能、交互验证
飞书未来无限大会（2023-2025）公开演讲	产品战略、AI 路线图
钉钉生态大会（2024-2025）公开演讲	产品战略、AI 路线图
建筑科技公司实习经历	工地场景观察（已脱敏）

附录 B：分析局限性

1. 本报告基于产品实测和个人观察，未包含大规模用户访谈和定量数据分析。
 2. AI 功能迭代极快，当前对比可能随版本更新而变化。
 3. 工地场景为实习期间对有限项目部的观察，不代表所有建筑企业。
-